

Contents

1	Introduzione e definizione del problema	3
1.1	Analisi del problema: Classificatori flat vs gerarchici	3
1.2	Obiettivo e impostazione dello studio	4
1.3	Struttura della tesi	4
2	Fondamenti teorici sulle architetture	6
2.1	Apprendimento multi-task gerarchico	6
2.2	Backbone per l'estrazione di feature	8
2.2.1	ResNet	8
2.2.2	Densenet	9
2.2.3	EfficientNet	10
2.3	Analisi comparativa	11
3	Pipeline sperimentale e metodologia	12
3.1	Dataset e task	12
3.2	Architettura multi-task	12
3.3	Procedura di training dei modelli	13
3.4	Degradazione controllata del training set	13
3.5	Metriche di valutazione	13
4	Risultati e discussione	14
4.1	Risultati Dataset Oxford-IIIT Pet	14
4.2	Risultati Dataset CIFAR-100	14
4.3	Discussione sui risultati ottenuti	14
5	Approccio prototypical network	15

5.1	Motivazione: il problema delle classi non viste	15
5.2	Prototypical network: principio di funzionamento	15
5.3	Setup e campionamento	15
5.4	Risultati ottenuti e discussione	15
6	Conclusione e sviluppi futuri	16
6.1	Sintesi e limiti dello studio	16
6.2	Sviluppi futuri	16

Capitolo 1

Introduzione e definizione del problema

1.1 Analisi del problema: Classificatori flat vs gerarchici

Il presente elaborato indaga la robustezza di classificatori gerarchici a due livelli, ove il primo livello specifica la superclasse di appartenenza, il secondo livello specifica la sottoclasse.

Tali modelli operano contemporaneamente a livello generale, definito *coarse*, e a livello specifico di sottoclasse, definito *fine*. La classificazione *coarse* richiede l'estrazione di feature semantiche globali. La classificazione *fine* è intrinsecamente più complessa: il modello deve individuare dettagli visivi altamente specifici e discriminativi tra soggetti affini, appartenenti allo stesso gruppo generale.

Un classificatore *flat*, diversamente da un classificatore gerarchico a due livelli, assegna l'immagine ad una delle possibili N classi disponibili senza sfruttare somiglianze o differenze nette tra elementi di classi diverse. Ogni classe è trattata come equidistante dalle altre, indipendentemente dal fatto che alcune categorie condividano caratteristiche visive affini. Per un classificatore *flat*, confondere un *labrador* ed un *golden retriever* costituisce un errore identico a confondere lo stesso *labrador* con un *merluzzo*.

Queste somiglianze sono sfruttate dai classificatori gerarchici a due livelli. Dal punto di vista architetturale, il modello diventa multi-task, in cui un backbone posto alla base estrae feature come bordi, texture, forme, mentre le due head distinte si specializzano una in classificazione *coarse* e l'altra in quella *fine*, utilizzando il backbone come base di partenza.

Si consideri, a titolo esemplificativo, un dataset strutturato su due livelli generali: Piante e Animali, articolati a loro volta in Arboree ed Erbacee la prima, Mammiferi e Pesci la seconda.

La distinzione della superclasse risulta un compito semplice: presentano caratteristiche completamente diverse tra loro, basti immaginare un pino e un cane.

La classificazione delle sottoclassi, invece, espone la rete a un compito complesso.

È il caso di specie come i *cetacei*: condividono caratteristiche fisiche dei *mammiferi* adat-

tati ad un ambiente acquatico che possono indurre il classificatore a confonderli con i *pesci*.

Tale criticità nella classificazione costituisce il punto di partenza per l'analisi sperimentale presentata. Per valutare la robustezza dei classificatori gerarchici, il presente studio introduce delle riduzioni controllate del dataset di addestramento, operando su due configurazioni distinte: drop a livello *coarse* (riduzione quantitativa del dataset, senza intaccare totalmente alcuna classe) e drop a livello *fine* (riduzione selettiva attraverso la rimozione completa di una o più classi).

1.2 Obiettivo e impostazione dello studio

L'obiettivo dello studio è analizzare la robustezza dei modelli in condizioni di disponibilità limitate di informazioni per l'addestramento.

Le due tipologie di drop sono introdotte a diversi livelli di intensità. Per ogni livello, valutiamo il comportamento della rete davanti ad uno stesso test set, il quale è fisso per tutti i modelli.

L'analisi non si limita a valutare le capacità del modello di riconoscere categorie presenti nel set di addestramento, ma si estende al caso critico in cui alcune classi sono escluse totalmente dal set di training. In questa condizione, il classificatore non può riconoscere correttamente la classe, dato che non ha mai visto quelle caratteristiche specifiche. E' necessario un approccio diverso, che sfrutti il modello non come semplice classificatore, ma come estrattore di feature: a partire da poche immagini di supporto (few-shot) è possibile verificare se quelle poche rappresentazioni assimilate alla rete siano sufficienti a discriminare e quindi a riconoscere categorie mai viste in addestramento.

La metodologia applicata e la pipeline adottata sono descritte nel dettaglio nei capitoli successivi.

1.3 Struttura della tesi

L'elaborato è organizzato come segue:

Il Capitolo 2 descrive la pipeline adottata, i dataset utilizzati, le tipologie di reti adottate, le definizioni e implementazioni dei drop a livello coarse e fine. Si conclude con la configurazione degli esperimenti condotto

Il Capitolo 3 presenta i risultati relativi all'impatto dei drop coarse e fine sulla performance

dei classificatori, analizzando (per ogni categoria e intensità di drop) l'effetto che questi provocano nelle diverse reti.

Il Capitolo 4 affronta il caso critico di classi totalmente escluse dal training set, illustrando l'approccio basato su reti prototipiche. In particolare, il capitolo mostra come un modello, pur non avendo mai osservato determinata classi in fase di training, sia comunque in grado di discriminare e di distinguerle da altre classi partendo da poche immagini di supporto

Il Capitolo 5 conclude la tesi riassumendo i risultati ottenuti, discutendo i limiti dello studio e possibili sviluppi futuri

Capitolo 2

Fondamenti teorici sulle architetture

2.1 Apprendimento multi-task gerarchico

Le reti neurali convoluzionali (CNN, *Convolutional Neural Networks*) sono le architetture standard per l'elaborazione dei segnali visivi. Si evolvono rispetto al *Multi-Layer Perceptron* per l'utilizzo di tensori multidimensionali ($H \times W \times C$), ove H indica l'altezza, W indica la larghezza e C indica il numero di canali. Tale rappresentazione permette di preservare informazioni come le relazioni spaziali tra pixel vicini durante l'elaborazione, attraverso tre principi fondamentali:

- L'uso dei campi ricettivi locali (local receptive fields)
- Condivisione dei pesi tra i layer (weight sharing)
- Equivarianza alla traslazione (se l'oggetto si sposta, anche la feature map si sposta)

Le reti raccolgono informazioni dall'immagine tramite l'utilizzo di filtri chiamati kernel. Questi analizzano una porzione locale di pixel adiacenti alla volta e producono un output associato a quella specifica posizione spaziale.

L'output è definito così:

- X : tensore di input;
- W : tensore dei pesi del kernel;
- k : raggio del kernel;
- C_{in} : numero di canali di input;
- b_{out} : bias associato al canale di uscita.

$$Y(i, j, c_{out}) = b_{out} + \sum_{c=1}^{C_{in}} \sum_{m=-k}^k \sum_{n=-k}^k X(i+m, j+n, c) W(m, n, c, c_{out}) \quad (2.1)$$

Ovvero, il valore di output Y nella posizione spaziale (i, j) e nel canale di uscita c_{out} è calcolato come somma pesata dei valori del tensore input X in un intorno locale in posizione (i, j) di dimensione $(2k + 1) \times (2k + 1)$. Ad esempio, per $k = 1$ si ottiene un kernel di dimensione 3×3 , mentre per $k = 2$ si ottiene un kernel di dimensione 5×5 .

L'operazione viene eseguita considerando tutti i canali di input. Ogni valore di X viene moltiplicato per il corrispondente peso W , appreso durante la fase di addestramento, e i prodotti ottenuti vengono sommati tra loro. Infine, al risultato viene aggiunto il termine di bias b_{out} associato al canale di uscita.

Questa formulazione spiega i principi elencati precedentemente. Il valore in posizione (i, j) dipende solo dalla porzione di input relativa a quella posizione, e non all'intera immagine come in un'architettura di rete MLP. Questo spiega il principio dei campi ricettivi locali. I pixel vicini sono più correlati tra loro, producono più informazioni se analizzate insieme tramite i kernel.

Il principio della condivisione dei pesi permette l'utilizzo di un insieme di pesi W per calcolare Y in ogni posizione (i, j) del tensore di output. Senza di esso si avrebbe un insieme di pesi indipendenti per ogni posizione, e ci troveremmo nel caso di layer fully-connected.

Dal principio della condivisione dei pesi si arriva al principio della equivarianza alla traslazione: lo stesso kernel W viene applicato ad ogni posizione (i, j) del tensore di input. Un pattern rilevato in (i, j) sarà rilevato alla stessa maniera anche in $(i + \Delta, j + \Delta)$, qualora l'oggetto che lo produce si spostasse di un Δ nell'immagine. Un semplice spostamento nell'input produce uno spostamento corrispondente nell'output, senza dover calibrare i pesi per riconoscerlo.

Ciò che si è appena descritto definisce una singola operazione di convoluzione. Nella pratica non è sufficiente ad estrarre informazioni utili alla classificazione. Una CNN utilizza una sequenza di queste operazioni in ciò che vengono chiamati layer. Ogni layer riceve in input il tensore prodotto dal layer precedente. Il tensore viene elaborato e diventa input di un nuovo layer. I primi layer catturano informazioni semplici, come bordi e texture. Gli strati più interni catturano pattern complessi e significativi.

Il numero di layer impilati si definisce profondità della rete.

Si può pensare che l'aumento della profondità permette al modello di apprendere rappresentazioni sempre più complesse e astratte. Nella pratica avviene il contrario. L'aumento della profondità causa il problema del vanishing gradient.

Durante l'addestramento, la rete apprende l'immagine bilanciando i propri pesi. Il gra-

diente della loss viene calcolato con la regola della catena, propagandosi all'indietro attraverso i layer successivi con la backpropagation. Ad ogni layer il gradiente viene moltiplicato per dei fattori. Spesso, questi sono minori di 1. Con tanti layer, il gradiente tende a ridursi esponenzialmente, fino a diventare trascurabile. Ciò provoca un aggiornamento dei pesi lento, nullo in casi estremi, causando un arresto dell'apprendimento.

Una rete profonda non migliora le prestazioni, ma le peggiora nettamente. Non è causato da un overfitting ma da una semplice difficoltà di ottimizzazione. Le reti utilizzate in questa tesi nascono, con strategie diverse tra loro, per affrontare questo limite.

2.2 Backbone per l'estrazione di feature

2.2.1 ResNet

ResNet (*Residual Network*), proposta da He et al. [1], introduce il concetto di *residual learning*, basato sull'utilizzo di connessioni residue (*skip connections*). L'obiettivo è facilitare l'ottimizzazione di reti neurali molto profonde e mitigare il problema della degradazione delle prestazioni che può manifestarsi all'aumentare della profondità della rete.

L'idea alla base di ResNet consiste nel modificare il modo in cui un blocco di layer apprende la trasformazione desiderata. Anziché apprendere direttamente una generica mappatura $H(x)$, il blocco viene addestrato ad apprendere la funzione residua $F(x)$, definita come la differenza tra la mappatura desiderata e l'input del blocco:

- x : input del blocco residuo;
- $H(x)$: mappatura desiderata, ovvero la trasformazione che il blocco dovrebbe apprendere;
- $F(x)$: funzione residua appresa dai layer del blocco;
- y : output del blocco residuo.

La funzione residua è quindi definita come:

$$F(x) = H(x) - x \quad (2.2)$$

da cui segue:

$$H(x) = F(x) + x \quad (2.3)$$

L'output del blocco può pertanto essere espresso come:

$$y = F(x) + x \quad (2.4)$$

Il termine x viene trasferito direttamente all'uscita mediante una connessione che bypassa uno o più layer del blocco. Tale collegamento prende il nome di *skip connection* e costituisce l'elemento caratterizzante delle architetture ResNet.

La presenza della *skip connection* consente al blocco di apprendere soltanto la trasformazione necessaria rispetto all'input. Nel caso limite in cui la mappatura desiderata sia prossima alla funzione identità, il residuo $F(x)$ tende a essere prossimo a zero. L'output del blocco può quindi essere approssimato come:

$$y \approx 0 + x = x \quad (2.5)$$

In questo modo, il blocco non è costretto ad apprendere una trasformazione completa quando questa non è necessaria, ma può concentrarsi sull'apprendimento delle modifiche da apportare all'input per ottenere la rappresentazione desiderata. Questo meccanismo facilita la propagazione dell'informazione attraverso reti profonde e rende più agevole l'ottimizzazione dei parametri durante l'addestramento.

2.2.2 Densenet

Densenet, proposta da Huang et al. [2], riprende l'idea delle connessioni tra layer introdotta da ResNet, ma la sviluppa in modo più radicale: mentre ResNet collega ogni layer solo al layer immediatamente successivo tramite connessioni residue, in DenseNet ogni layer è connesso a *tutti* i layer successivi all'interno dello stesso blocco (denominato *dense block*).

Concretamente, l'input di ogni layer è dato dalla concatenazione delle feature map prodotte da tutti i layer precedenti nel blocco, anziché dalla sola somma usata da ResNet.

A titolo esemplificativo, supponiamo un blocco formato da 10 layer. Il primo layer riceve l'input del blocco e produce un output x_1 . Il secondo layer riceve x_1 e produce x_2 . Il terzo layer riceve la concatenazione di x_1 e x_2 , e così via: l' n -esimo layer riceve la concatenazione degli output di tutti gli $n - 1$ layer precedenti. Questo schema fa sì che il numero di connessioni cresca quadraticamente con la profondità del blocco (per un blocco di L layer si hanno $L(L + 1)/2$ connessioni), favorendo il riutilizzo delle feature e mitigando il problema del vanishing gradient.

Formalmente, l'output dell' ℓ -esimo layer è dato da $x_\ell = H_\ell([x_0, x_1, \dots, x_{\ell-1}])$, dove $[x_0, x_1, \dots, x_{\ell-1}]$ indica la concatenazione delle feature map prodotte dai layer precedenti e H_ℓ è una funzione composta (tipicamente Batch Normalization, ReLU e convoluzione). Ogni layer produce un numero fisso di nuove feature map, detto *growth rate* (k), che determina quanto rapidamente cresce il numero di canali all'interno del blocco.

Per evitare che questa crescita diventi insostenibile, tra un dense block e il successivo vengono inseriti dei *transition layer*, che riducono sia la risoluzione (tramite pooling) sia il numero di canali (tramite una convoluzione 1×1).

2.2.3 EfficientNet

EfficientNet, proposta da Tan e Le [5], nasce con l'obiettivo di massimizzare l'accuracy a parità di budget computazionale. A differenza degli approcci tradizionali, che scalano una sola dimensione della rete (ad esempio aumentando la risoluzione dell'input o semplicemente la profondità della rete), gli autori citati osservano che le tre dimensioni di scaling non sono indipendenti: bilanciarle tutte, contemporaneamente, produce risultati migliori rispetto a scalare una sola.

Le tre dimensioni in questione sono:

- **Profondità**: numero di layer della rete;
- **Larghezza**: numero di canali per layer;
- **Risoluzione**: dimensione dell'immagine in input.

EfficientNet scala queste dimensioni tramite il *compound scaling method*: dato un coefficiente ϕ che rappresenta le risorse computazionali disponibili, profondità, larghezza e risoluzione vengono scalate secondo

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \quad \text{con} \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \quad (2.6)$$

dove α, β, γ sono costanti trovate tramite grid search su un modello base (*EfficientNet-B0*, ottenuto tramite *NAS*, Neural Architecture Search). Queste costanti stabiliscono come ripartire l'aumento di risorse tra le tre dimensioni; è invece il coefficiente ϕ , scelto dall'utente, a determinare quante risorse complessive si vogliono impiegare. Ogni incremento di ϕ porta a un raddoppio approssimativo dei FLOPs della rete (*floating point operations*, il numero di operazioni in virgola mobile richieste per un'inferenza), pari a 2^ϕ .

2.3 Analisi comparativa

Le architetture presentate affrontano da diverse prospettive il problema di rendere le reti neurali sempre più profonde ed efficienti:

- **ResNet**: introduce l'apprendimento residuo tramite le skip connection, mitigando il problema del vanishing gradient;
- **DenseNet**: sviluppa l'idea di connessioni tra layer, favorendo il riutilizzo delle feature tramite connessioni dense
- **EfficientNet**: si concentra e ottimizza il bilancio tra profondità, larghezza e risoluzione dell'immagine, per massimizzare l'accuracy con un budget di potenza di calcolo.

Una delle domande a cui questa tesi sperimentale tenta di rispondere è: come reagiscono queste diverse architetture a differenti tipologie di perdite controllate di informazioni nel training set?

Capitolo 3

Pipeline sperimentale e metodologia

3.1 Dataset e task

Per realizzare la tesi sono stati utilizzati due dataset:

- *Oxford-IIIT Pet* [4], composto da immagini di cani e gatti suddivise in 37 classi. Il dataset comprende un file di annotazione che associa ad ogni immagine una tupla (`image_path`, `micro_label` e `macro_label`), ove:
 - `micro_label` identifica la razza tra le 37 disponibili;
 - `macro_label` identifica la specie, ovvero cane o gatto.

Il dataset è diviso in tre sottoinsiemi: training (70%), validazione (15%) e test (15%). La suddivisione è realizzata tramite uno split in modo da preservare la distribuzione equa delle 37 classi.

- *CIFAR-100* [3], composto da 100 classi raggruppate in 20 superclassi. Ogni super-classe comprende 5 classi, per un totale di 100 fine label. Per contenere il costo computazionale degli esperimenti, è stato estratto inizialmente un sottoinsieme di 50 campioni per ciascuna classe. Successivamente la quantità è stata aumentata a 200 campioni per classe, poiché la quantità iniziale produceva un risultato caratterizzato da variabilità e rumore.

Il campionamento è stato effettuato prima della suddivisione stratificata del dataset, secondo la proporzione 70%/15%/15% tra training, validation e test.

3.2 Architettura multi-task

Delle famiglie architetture presentate nel capitolo 2.2, sono state selezionate quattro varianti come backbone per gli esperimenti: **ResNet18**, **ResNet50**, **DenseNet121** ed **EfficientNet-B0**.

L'integrazione del task multi-label sui backbone descritti nel capitolo 2.2 segue uno schema standard, noto come *hard parameter sharing*: un unico backbone convoluzionale è utilizzato da due head di classificazione indipendenti.

Il backbone elabora l'immagine fino al GAP¹, producendo un vettore di embedding la cui dimensione dipende dall'architettura:

- 512 per ResNet18;
- 2048 per ResNet50;
- 1024 per DenseNet121;
- 1280 per EfficientNet-B0.

Questo vettore è utilizzato in parallelo da entrambe le head.

Le due head sono costituite da un singolo layer lineare (`Linear(feature_dim, num_classes)`).

Le proiezioni delle due teste sono, rispettivamente, su 37 classi (razza) e 2 classi (specie) per Oxford-IIIT Pet; su 100 classi (*fine*) e 20 classi (*coarse*) per CIFAR-100.

La funzione di loss è una combinazione pesata delle due cross-entropy, tramite i coefficienti α e β :

$$\mathcal{L}_{\text{tot}} = \alpha \cdot \mathcal{L}_{\text{micro}} + \beta \cdot \mathcal{L}_{\text{macro}} \quad (3.1)$$

con $\alpha = 1.0$ e $\beta = 0.5$: il task fine-grained (micro) pesa dunque il doppio rispetto al task coarse (macro) nella loss complessiva.

3.3 Procedura di training dei modelli

3.4 Degradazione controllata del training set

3.5 Metriche di valutazione

¹ GAP: *Global Average Pooling*. Operazione applicata al termine del backbone che riduce ciascuna mappa di feature a un singolo valore medio, trasformando così il volume di feature tridimensionale in un vettore monodimensionale utilizzabile dalle head di classificazione.

Capitolo 4

Risultati e discussione

4.1 Risultati Dataset Oxford-IIIT Pet

4.2 Risultati Dataset CIFAR-100

4.3 Discussione sui risultati ottenuti

Capitolo 5

Approccio prototypical network

5.1 Motivazione: il problema delle classi non viste

5.2 Prototypical network: principio di funzionamento

5.3 Setup e campionamento

5.4 Risultati ottenuti e discussione

Capitolo 6

Conclusione e sviluppi futuri

6.1 Sintesi e limiti dello studio

6.2 Sviluppi futuri

Bibliography

- [1] Kaiming He et al. ``Deep Residual Learning for Image Recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.
- [2] Gao Huang et al. *Densely Connected Convolutional Networks*. 2018. arXiv: 1608.06993 [cs.CV]. URL: <https://arxiv.org/abs/1608.06993>.
- [3] Alex Krizhevsky and Geoffrey Hinton. *Learning multiple layers of features from tiny images*. Tech. rep. 0. Toronto, Ontario: University of Toronto, 2009. URL: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [4] Omkar M. Parkhi et al. ``Cats and Dogs". In: *IEEE Conference on Computer Vision and Pattern Recognition*. 2012.
- [5] Mingxing Tan and Quoc V. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. 2020. arXiv: 1905.11946 [cs.LG]. URL: <https://arxiv.org/abs/1905.11946>.